



Complete Tree & Graph Guide



Interview Ready Notes • Code Snippets • Theory & Complexity

PART 1: TREES

1 Binary Tree Basics

A hierarchical structure where every node has at most 2 children. This is the foundation for Heaps, BSTs, and Tries.

Node Structure:

```
class TreeNode {  
    int val;  
    TreeNode left, right;  
  
    TreeNode(int val) {  
        this.val = val;  
    }  
}
```

Example Structure:

```
      1  
     / \  
    2   3  
   / \   \  
  4  5  6
```

Tree Traversals (The Interview Core)

💡 Why this matters:

If you can't traverse a tree, you can't solve any tree problem. Memorize these four patterns.

1. Inorder (Left → Root → Right)

```
void inorder(TreeNode root) {
    if (root == null) return;
    inorder(root.left);
    System.out.print(root.val + " ");
    inorder(root.right);
}
// Output: 4 2 5 1 3 6 | Key Fact: Returns sorted data in a BST!
```

2. Preorder (Root → Left → Right)

```
void preorder(TreeNode root) {
    if (root == null) return;
    System.out.print(root.val + " ");
    preorder(root.left);
    preorder(root.right);
}
// Output: 1 2 4 5 3 6 | Key Fact: Useful for cloning/copying a tree.
```

3. Postorder (Left → Right → Root)

```
void postorder(TreeNode root) {
    if (root == null) return;
    postorder(root.left);
    postorder(root.right);
    System.out.print(root.val + " ");
}
// Output: 4 5 2 6 3 1 | Key Fact: Useful for deleting a tree (delete children first).
```

4. Level Order (BFS using Queue)

```
void levelOrder(TreeNode root) {  
    if (root == null) return;  
    Queue<TreeNode> q = new LinkedList<>();  
    q.offer(root);  
  
    while (!q.isEmpty()) {  
        TreeNode node = q.poll();  
        System.out.print(node.val + " ");  
        if (node.left != null) q.offer(node.left);  
        if (node.right != null) q.offer(node.right);  
    }  
}  
  
// Output: 1 2 3 4 5 6
```



These are standard standard algorithmic building blocks.

Height, Count & Diameter

```
// 1. HEIGHT - O(N)
int height(TreeNode root) {
    if (root == null) return -1;
    return 1 + Math.max(height(root.left), height(root.right));
}

// 2. COUNT NODES - O(N)
int countNodes(TreeNode root) {
    if (root == null) return 0;
    return 1 + countNodes(root.left) + countNodes(root.right);
}

// 3. DIAMETER (Longest path between any two nodes) - O(N)
int diameter(TreeNode root) {
    int[] max = {0};
    diameterHelper(root, max);
    return max[0];
}

int diameterHelper(TreeNode root, int[] max) {
    if (root == null) return 0;
    int left = diameterHelper(root.left, max);
    int right = diameterHelper(root.right, max);
    max[0] = Math.max(max[0], left + right); // Update global max
    return 1 + Math.max(left, right); // Return height
}
```

LCA, Balanced Check & Mirror

```
// 4. LOWEST COMMON ANCESTOR - O(N)
TreeNode lca(TreeNode root, int p, int q) {
    if (root == null || root.val == p || root.val == q) return root;

    TreeNode left = lca(root.left, p, q);
    TreeNode right = lca(root.right, p, q);

    if (left != null && right != null) return root;
    return (left != null) ? left : right;
}
```

```
}
```

```
// 5. CHECK BALANCED -  $O(N)$ 
```

```
boolean isBalanced(TreeNode root) {  
    return checkHeight(root) != -1;  
}
```

```
int checkHeight(TreeNode root) {  
    if (root == null) return 0;  
    int left = checkHeight(root.left);  
    if (left == -1) return -1;  
    int right = checkHeight(root.right);  
    if (right == -1) return -1;  
  
    if (Math.abs(left - right) > 1) return -1;  
    return 1 + Math.max(left, right);  
}
```

```
// 6. MIRROR/INVERT TREE -  $O(N)$  - (The famous Homebrew/Google question)
```

```
TreeNode invertTree(TreeNode root) {  
    if (root == null) return null;  
    TreeNode left = invertTree(root.left);  
    TreeNode right = invertTree(root.right);  
    root.left = right;  
    root.right = left;  
    return root;  
}
```

2 Binary Search Tree (BST) ★

✔ BST Properties:

Left subtree < Root < Right subtree. **Inorder Traversal is always sorted.**

```
class BST {
    // 1. SEARCH - O(H)
    boolean search(TreeNode root, int key) {
        if (root == null) return false;
        if (root.val == key) return true;
        return (key < root.val) ? search(root.left, key) : search(root.right, key);
    }

    // 2. INSERT - O(H)
    TreeNode insert(TreeNode root, int val) {
        if (root == null) return new TreeNode(val);
        if (val < root.val) root.left = insert(root.left, val);
        else if (val > root.val) root.right = insert(root.right, val);
        return root;
    }

    // 3. DELETE - O(H)
    TreeNode delete(TreeNode root, int key) {
        if (root == null) return null;

        if (key < root.val) root.left = delete(root.left, key);
        else if (key > root.val) root.right = delete(root.right, key);
        else {
            // Node found
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;

            // Two children: replace with inorder successor (min of right subtree)
            TreeNode min = findMin(root.right);
            root.val = min.val;
            root.right = delete(root.right, min.val);
        }
        return root;
    }

    TreeNode findMin(TreeNode root) {
        while (root.left != null) root = root.left;
        return root;
    }
}
```

```

}

// 4. VALIDATE BST - O(N)
boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

boolean validate(TreeNode root, long min, long max) {
    if (root == null) return true;
    if (root.val <= min || root.val >= max) return false;
    return validate(root.left, min, root.val) && validate(root.right, root.val,
max);
}

// 5. KTH SMALLEST ELEMENT - O(K)
int kthSmallest(TreeNode root, int k) {
    Stack<TreeNode> stack = new Stack<>();
    TreeNode curr = root;
    int count = 0;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        if (++count == k) return curr.val;
        curr = curr.right;
    }
    return -1;
}
}

```

3 Heap (Min/Max) & 4 Trie (Prefix Tree)

Essential data structures for Priority Queues and String Search optimizations.

Min Heap Implementation

```
class MinHeap {
    int[] heap; int size;
    // Parent: (i-1)/2 | Left: 2*i+1 | Right: 2*i+2

    // INSERT - O(Log N) - Add to end, bubble up
    void insert(int val) {
        heap[size] = val;
        int i = size++;
        while (i > 0 && heap[i] < heap[(i-1)/2]) {
            swap(i, (i-1)/2);
            i = (i-1)/2;
        }
    }

    // EXTRACT MIN - O(Log N) - Swap root with last, bubble down
    int extractMin() {
        int min = heap[0];
        heap[0] = heap[--size];
        heapifyDown(0);
        return min;
    }

    void heapifyDown(int i) {
        int smallest = i, l = 2*i+1, r = 2*i+2;
        if (l < size && heap[l] < heap[smallest]) smallest = l;
        if (r < size && heap[r] < heap[smallest]) smallest = r;
        if (smallest != i) {
            swap(i, smallest);
            heapifyDown(smallest);
        }
    }
}
```

Trie Implementation (Autocomplete)

```
class Trie {
    class TrieNode {
```



```

        TrieNode[] children = new TrieNode[26];
        boolean isEndOfWord = false;
    }
    TrieNode root = new TrieNode();

    // INSERT - O(L)
    void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new TrieNode();
            node = node.children[idx];
        }
        node.isEndOfWord = true;
    }

    // SEARCH - O(L)
    boolean search(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children[c - 'a'] == null) return false;
            node = node.children[c - 'a'];
        }
        return node.isEndOfWord;
    }

    // DELETE - Recursion needed to prune unused branches
    boolean delete(String word) { return deleteHelper(root, word, 0); }

    boolean deleteHelper(TrieNode node, String word, int idx) {
        if (idx == word.length()) {
            if (!node.isEndOfWord) return false;
            node.isEndOfWord = false;
            return hasNoChildren(node);
        }
        int charIdx = word.charAt(idx) - 'a';
        TrieNode child = node.children[charIdx];
        if (child == null) return false;

        boolean shouldDeleteChild = deleteHelper(child, word, idx+1);
        if (shouldDeleteChild) {
            node.children[charIdx] = null;
            return !node.isEndOfWord && hasNoChildren(node);
        }
        return false;
    }
}

```

```
boolean hasNoChildren(TrieNode node) {  
    for (TrieNode c : node.children) if (c != null) return false;  
    return true;  
}  
}
```

PART 2: GRAPHS

1 Graph Basics & Representations

💡 Representation Strategy:

Use **Adjacency List** for 95% of problems (Saves space $O(V+E)$). Use **Matrix** only for dense graphs ($O(V^2)$).

```
// 1. ADJACENCY MATRIX -  $O(V^2)$  Space
```

```
class GraphMatrix {
    int[][] matrix;
    void addEdge(int u, int v) {
        matrix[u][v] = 1;
        matrix[v][u] = 1; // Undirected
    }
}
```

```
// 2. ADJACENCY LIST -  $O(V+E)$  Space (Recommended)
```

```
class Graph {
    List<List<Integer>> adj;
    int vertices;

    Graph(int v) {
        vertices = v;
        adj = new ArrayList<>();
        for (int i = 0; i < v; i++) adj.add(new ArrayList<>());
    }

    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u);
    }
}
```

2 Graph Traversals (BFS & DFS)

BFS (Breadth-First Search)

Uses a **Queue**. Good for shortest paths in unweighted graphs.

```
void bfs(int start) {
    boolean[] visited = new boolean[vertices];
    Queue<Integer> queue = new LinkedList<>();

    visited[start] = true;
    queue.offer(start);

    while (!queue.isEmpty()) {
        int node = queue.poll();
        System.out.print(node + " ");

        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
}
```

DFS (Depth-First Search)

Uses **Recursion (Stack)**. Good for exhaustive search and cycle detection.

```
void dfs(int node, boolean[] visited) {
    visited[node] = true;
    System.out.print(node + " ");

    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) {
            dfs(neighbor, visited);
        }
    }
}
```

Dijkstra (Weighted, No Negative Edges) - $O(E \log V)$

```
int[] dijkstra(List<List<Node>> graph, int src, int V) {
    int[] dist = new int[V];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    PriorityQueue<Node> pq = new PriorityQueue<>((a, b) -> a.dist - b.dist);
    pq.offer(new Node(src, 0));

    while (!pq.isEmpty()) {
        Node curr = pq.poll();
        int u = curr.vertex;

        for (Node neighbor : graph.get(u)) {
            if (dist[u] + neighbor.dist < dist[neighbor.vertex]) {
                dist[neighbor.vertex] = dist[u] + neighbor.dist;
                pq.offer(new Node(neighbor.vertex, dist[neighbor.vertex]));
            }
        }
    }
    return dist;
}
```

Bellman-Ford (Detects Negative Cycles) - $O(VE)$

```
int[] bellmanFord(int V, List<int[]> edges, int src) {
    int[] dist = new int[V];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // Relax all edges V-1 times
    for (int i = 0; i < V - 1; i++) {
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
}
```

```

// Check for negative weight cycle
for (int[] edge : edges) {
    if (dist[edge[0]] + edge[2] < dist[edge[1]]) return null; // Cycle detected
}
return dist;
}

```

Floyd-Warshall (All Pairs Shortest Path) - $O(V^3)$

```

void floydWarshall(int[][] graph, int V) {
    int[][] dist = new int[V][V];
    for(int i=0; i<V; i++) {
        for (int k = 0; k < V; k++) {
            for (int i = 0; i < V; i++) {
                for (int j = 0; j < V; j++) {
                    if (dist[i][k] != Integer.MAX_VALUE && dist[k][j] !=
Integer.MAX_VALUE &&
                        dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }
    }
}

```

Kruskal's Algorithm (MST using Union-Find)

```
List<Edge> kruskalMST(List<Edge> edges, int V) {
    Collections.sort(edges); // Sort edges by weight
    UnionFind uf = new UnionFind(V);
    List<Edge> mst = new ArrayList<>();

    for (Edge edge : edges) {
        if (uf.union(edge.src, edge.dest)) {
            mst.add(edge);
            if (mst.size() == V - 1) break;
        }
    }
    return mst;
}
```

Topological Sort (Kahn's Algorithm / BFS)

Only works on **DAGs** (Directed Acyclic Graphs). Used for dependency resolution.

```
List<Integer> kahnTopologicalSort() {
    int[] indegree = new int[vertices];
    for (int u = 0; u < vertices; u++)
        for (int v : adj.get(u)) indegree[v]++;

    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < vertices; i++) if (indegree[i] == 0) q.offer(i);

    List<Integer> result = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        result.add(u);
        for (int v : adj.get(u)) {
            indegree[v]--;
            if (indegree[v] == 0) q.offer(v);
        }
    }
}
```

```
return result; // If result.size() != V, there's a cycle  
}
```


Cycle Detection (Directed Graph)

Uses recursionStack to track current path nodes.

```
boolean hasCycleDirected() {
    boolean[] visited = new boolean[vertices];
    boolean[] recStack = new boolean[vertices];

    for (int i = 0; i < vertices; i++)
        if (checkCycle(i, visited, recStack)) return true;
    return false;
}

boolean checkCycle(int v, boolean[] visited, boolean[] recStack) {
    if (recStack[v]) return true;
    if (visited[v]) return false;

    visited[v] = true; recStack[v] = true;
    for (int n : adj.get(v))
        if (checkCycle(n, visited, recStack)) return true;

    recStack[v] = false; // Backtrack
    return false;
}
```

Strongly Connected Components (Kosaraju's Algo)

```
void printSCCs() {
    Stack<Integer> stack = new Stack<>();
    boolean[] visited = new boolean[vertices];

    // 1. Fill Stack (Finish time)
    for (int i=0; iif(!visited[i]) fillOrder(i, visited, stack);

    // 2. Transpose Graph
    Graph trans = getTranspose();

    // 3. DFS on Transpose
    Arrays.fill(visited, false);
    while (!stack.isEmpty()) {
        int v = stack.pop();
```

```
        if (!visited[v]) {
            trans.dfs(v, visited); // One SCC
            System.out.println();
        }
    }
}
```

 Ultimate Complexity Cheat Sheet

Algorithm	Time Complexity	Space	Use Case
BFS / DFS	$O(V + E)$	$O(V)$	Traversal, Cycle, Connectivity
Dijkstra	$O(E \log V)$	$O(V)$	Shortest Path (Weighted +)
Bellman-Ford	$O(VE)$	$O(V)$	Shortest Path (Negative weights)
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All-Pairs Shortest Path
Kruskal (MST)	$O(E \log E)$	$O(V)$	Min Spanning Tree
Topological Sort	$O(V + E)$	$O(V)$	Dependency Resolution